PUCPR

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ

DATA ANALYTICS / PROJETO BASEADO EM PROBLEMAS

RA1 e RA2

Ingestão, preparação e streaming de dados

Notebook Jupyter da entrega

Projeto O que determina o desempenho e o valor dos veículos do GTA V?

Curso e turma BSI • 6U

Professora Marina de Lara

Equipe Rogerio de Abreu Mar, Felipe Da Veiga Gomes, Logan Borges, Alex Narok Stavasz

Repositório https://github.com/ODevAbreu/DataAnalytics

Entrega Curitiba • 21/10/2026

Este documento é o notebook Jupyter exportado, com código, saídas e explicações. **RA1** (seções 1 a 6): ingestão e preparação dos dados. **RA2** (seções 7 a 12): processamento em streaming com Spark Structured Streaming e janelas temporais.

RA1 / 01

Definição do problema e perguntas analíticas

Contexto. GTA V possui um catálogo com centenas de veículos (carros, motos, aeronaves, barcos) adicionados ao longo de ~10 anos de atualizações (DLCs). Cada veículo tem atributos de desempenho (velocidade máxima, aceleração, frenagem, tração), atributos físicos de *handling* (massa, força do motor, marchas, freio) e um valor monetário no jogo.

Problema. Para jogadores que escolhem em qual veículo investir, e para quem estuda balanceamento de jogos, não é claro **o que determina o desempenho e o valor de um veículo**, nem se os veículos lançados nas atualizações mais recentes são sistematicamente melhores (*power creep*).

Perguntas analíticas (respondidas nos próximos RAs; o RA1 prepara os dados para elas):

1. **P1 – Desempenho por categoria:** como velocidade máxima, aceleração e frenagem variam entre **classes** (SUPER, SPORT, MUSCLE…) e **tipos** (CAR, BIKE, PLANE…)?
2. **P2 – Valor × desempenho:** existe relação entre o **valor monetário** e as métricas de desempenho? Quais classes/fabricantes oferecem a melhor relação desempenho/valor?
3. **P3 – Física × desempenho:** atributos de *handling* (massa, força do motor, velocidade de projeto, nº de marchas, força de freio) explicam as métricas de desempenho?
4. **P4 – Fabricantes:** quais fabricantes concentram os veículos mais rápidos e mais caros?
5. **P5 – Evolução por DLC (*power creep*):** veículos adicionados por DLCs têm desempenho e valor superiores aos do jogo base (`TitleUpdate`)?

RA1 / 02

Fontes de dados

Item	`vehicles.json`	`vehicleHandlings.json`
Origem	Repositório público loopgazer/GTA.V.Dataset (fork de DurtyFree/gta-v-data-dumps) — dados extraídos automaticamente dos arquivos do jogo, versão **v2944 (Online 1.67)**	mesmo repositório
Commit fixado	`8ff8268bac0d86a7e384f8b66d6b8dc5dd7c1eea` (21/06/2023)	idem
Formato	JSON (array de objetos, aninhado)	JSON (array de objetos, aninhado)
Registros	815 veículos	747 perfis de *handling*
Granularidade	1 linha = 1 modelo de veículo	1 linha = 1 perfil de dirigibilidade (compartilhável entre veículos)
Principais atributos	`Name`, `DisplayName`, `Manufacturer`, `Class`, `Type`, `DlcName`, `HandlingId`, `Seats`, `MonetaryValue`, `Price`, `MaxSpeed`, `Acceleration`, `MaxBraking`, `MaxTraction`, `Agility`, `Flags`, dimensões	`Id`, `Mass`, `InitialDriveForce`, `InitialDriveMaxFlatVel`, `InitialDriveGears`, `DriveBiasFront`, `BrakeForce`, `TractionCurveMax/Min`, `SuspensionForce`, multiplicadores de dano
Relação com o problema	Fornece categorias (P1, P4, P5), valor (P2) e métricas de desempenho (P1–P5)	Fornece as variáveis físicas que explicam o desempenho (P3)

Chave de ligação: `vehicles.HandlingId` ↔ `vehicleHandlings.Id` (relação N:1 — vários veículos podem compartilhar o mesmo *handling*).

Os demais arquivos do repositório (sons, animações, cutscenes etc.) são listas de referência para *modding*, sem atributos analíticos relevantes ao problema, e por isso ficaram fora do escopo.

RA1 / 03

Ingestão dos dados

3.1 Ambiente

O Colab já traz o PySpark instalado; a célula abaixo só instala caso não esteja disponível (ex.: execução local).

```
import importlib.util
import subprocess
import sys

if importlib.util.find_spec("pyspark") is None:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "pyspark==3.5.3"])
```

```
import hashlib
import json
import urllib.request
from datetime import datetime, timezone
from pathlib import Path

from pyspark.sql import SparkSession, Window
from pyspark.sql import functions as F
from pyspark.sql import types as T

spark = (
    SparkSession.builder
    .appName("RA1-GTAV-Veiculos")
    .master("local[*]")
    .config("spark.sql.session.timeZone", "UTC")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")
print("Spark", spark.version, "| Python", sys.version.split()[0])
```

```
26/10/21 01:01:57 WARN Utils: Your hostname, vm resolves to a loopback address: 127.0.0.1; using
192.0.2.2 instead (on interface eth0)
26/10/21 01:01:57 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/21 01:01:57 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable
Spark 3.5.3 | Python 3.12.3
```

3.2 Parâmetros da ingestão

A URL aponta para um **commit fixo**, não para o branch: se o repositório de origem for atualizado, o notebook continua lendo exatamente os mesmos dados (reprodutibilidade).

```
REPO = "loopgazer/GTA.V.Dataset"
COMMIT = "8ff8268bac0d86a7e384f8b66d6b8dc5dd7c1eea"
ARQUIVOS = ["vehicles.json", "vehicleHandlings.json"]

BASE = Path("/content/ra1") if Path("/content").exists() else Path("ra1")
RAW = BASE / "data" / "raw"
PROCESSED = BASE / "data" / "processed"
RAW.mkdir(parents=True, exist_ok=True)
PROCESSED.mkdir(parents=True, exist_ok=True)
print("Dados brutos em:", RAW.resolve())
```

```
Dados brutos em: /home/claude/work/ra1/data/raw
```

3.3 Download dos arquivos brutos (camada *raw*)

Cada arquivo é baixado uma vez, e registramos tamanho e hash SHA-256 para provar que os dados não mudaram entre execuções.

```
def baixar(arquivo):
    url = f"https://raw.githubusercontent.com/{REPO}/{COMMIT}/{arquivo}"
    destino = RAW / arquivo
    if not destino.exists():
        urllib.request.urlretrieve(url, destino)
    return {
        "arquivo": arquivo,
        "url": url,
        "bytes": destino.stat().st_size,
        "sha256": hashlib.sha256(destino.read_bytes()).hexdigest(),
    }


log_ingestao = {
    "executado_em": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    "repositorio": REPO,
    "commit": COMMIT,
    "arquivos": [baixar(a) for a in ARQUIVOS],
}
(RAW / "ingestao_log.json").write_text(json.dumps(log_ingestao, indent=2), encoding="utf-8")

for a in log_ingestao["arquivos"]:
    print(f"{a['arquivo']:<24} {a['bytes']:>10,} bytes  sha256={a['sha256'][:16]}...")
```

```
vehicles.json             8,794,733 bytes  sha256=c06e2ea75b66027f...
vehicleHandlings.json     1,184,393 bytes  sha256=96a02ed7aed9ecd2...
```

3.4 Leitura com PySpark

Os arquivos são um **array JSON multilinha**, por isso `multiLine=True`. O modo `FAILFAST` faz a leitura falhar caso exista algum registro malformado (em vez de gerar linhas corrompidas silenciosamente). A coluna técnica `_ordem_origem` guarda a posição do registro no arquivo original (cada arquivo vira uma única partição, então a ordem é preservada) — ela será usada no tratamento de duplicatas.

```
def ler_json(arquivo):
    return (
        spark.read
        .option("multiLine", True)
        .option("mode", "FAILFAST")
        .json(str(RAW / arquivo))
        .withColumn("_ordem_origem", F.monotonically_increasing_id())
    )


veh_raw = ler_json("vehicles.json").cache()
hdl_raw = ler_json("vehicleHandlings.json").cache()

print(f"vehicles.json         -> {veh_raw.count():>4} linhas x {len(veh_raw.columns) - 1} colunas (+
_ordem_origem)")
print(f"vehicleHandlings.json -> {hdl_raw.count():>4} linhas x {len(hdl_raw.columns) - 1} colunas (+
_ordem_origem)")
```

```
26/10/21 01:02:05 WARN SparkStringUtils: Truncated the string representation of a plan since it was
too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.

vehicles.json         ->  815 linhas x 53 colunas (+ _ordem_origem)
vehicleHandlings.json ->  747 linhas x 48 colunas (+ _ordem_origem)
```

RA1 / 04

Compreensão e avaliação da qualidade dos dados

Funções auxiliares usadas nas análises:

```
def tabela(linhas, colunas, n=100):
    # Exibe uma lista de tuplas como DataFrame Spark.
    if not linhas:
        print("(nenhum registro)")
        return
    spark.createDataFrame(linhas, colunas).show(n, truncate=False)


def tipos(df):
    return [(f.name, f.dataType.simpleString()[:70]) for f in df.schema.fields if not
f.name.startswith("_")]


def perfil_ausentes(df):
    # Conta nulos; para strings, conta também valores vazios (""), que são ausências disfarçadas.
    exprs = []
    for f in df.schema.fields:
        if f.name.startswith("_"):
            continue
        cond = F.col(f.name).isNull()
        if isinstance(f.dataType, T.StringType):
            cond = cond | (F.trim(F.col(f.name)) == "")
        exprs.append(F.sum(F.when(cond, 1).otherwise(0)).alias(f.name))
    total = df.count()
    contagem = df.agg(*exprs).first().asDict()
    return [(c, int(n), round(100 * n / total, 1)) for c, n in contagem.items() if n > 0]
```

4.1 Estrutura e tipos

O Spark inferiu os tipos a partir do JSON. Verificamos se os numéricos vieram como número e quais colunas são aninhadas (struct/array) e precisarão ser achatadas.

```
print("== vehicles.json ==")
tabela(tipos(veh_raw), ["coluna", "tipo"])
```

```
== vehicles.json ==
+----------------------+--------------------------------------------------------------------------+
|coluna                |tipo                                                                      |
+----------------------+--------------------------------------------------------------------------+
|Acceleration          |double                                                                    |
|AdditionalTrailers    |array<string>                                                             |
|Agility               |double                                                                    |
|Bones                 |array<struct<BoneId:bigint,BoneIndex:bigint,BoneName:string>>             |
|BoundingCenter        |struct<X:double,Y:double,Z:double>                                        |
|BoundingSphereRadius  |double                                                                    |
|Class                 |string                                                                    |
|ClassId               |bigint                                                                    |
|DashboardType         |string                                                                    |
|DefaultBodyHealth     |double                                                                    |
|DefaultColors         |array<struct<DefaultDashboardColor:bigint,DefaultInteriorColor:bigint,|
|DefaultHorn           |bigint                                                                    |
|DefaultHornVariation  |bigint                                                                    |
|DimensionsMax         |struct<X:double,Y:double,Z:double>                                        |
|DimensionsMin         |struct<X:double,Y:double,Z:double>                                        |
|DirtLevelMax          |double                                                                    |
|DirtLevelMin          |double                                                                    |
|DisplayName           |struct<English:string,French:string,German:string,Hash:bigint,Italian:|
|DlcName               |string                                                                    |
|Extras                |array<bigint>                                                             |
|Flags                 |array<string>                                                             |
|HandlingId            |string                                                                    |
|HasArmoredWindows     |boolean                                                                   |
```

```
|HasConvertibleRoof     |boolean                                                                 |
|HasKers                |boolean                                                                 |
|HasParachute           |boolean                                                                 |
|HasSirens              |boolean                                                                 |
|Hash                   |bigint                                                                  |
|HexHash                |string                                                                  |
|LayoutId               |string                                                                  |
|Manufacturer           |string                                                                  |
|ManufacturerDisplayName|struct<English:string,French:string,German:string,Hash:bigint,Italian:|
|MaxBraking             |double                                                                  |
|MaxBrakingMods         |double                                                                  |
|MaxKnots               |double                                                                  |
|MaxSpeed               |double                                                                  |
|MaxTraction            |double                                                                  |
|ModKits                |array<string>                                                           |
|MonetaryValue          |bigint                                                                  |
|MoveResistance         |double                                                                  |
|Name                   |string                                                                  |
|PlateType              |string                                                                  |
|Price                  |bigint                                                                  |
|RequiredExtras         |array<bigint>                                                           |
|Rewards                |array<string>                                                           |
|Seats                  |bigint                                                                  |
|SignedHash             |bigint                                                                  |
|SpawnFrequency         |double                                                                  |
|Trailers               |array<string>                                                           |
|Type                   |string                                                                  |
|Weapons                |array<string>                                                           |
|WheelType              |string                                                                  |
|WheelsCount            |bigint                                                                  |
+-----------------------+------------------------------------------------------------------------+
```

```
print("== vehicleHandlings.json ==")
tabela(tipos(hdl_raw), ["coluna", "tipo"])
```

```
== vehicleHandlings.json ==
+-----------------------------+---------------------------------+
|coluna                       |tipo                             |
+-----------------------------+---------------------------------+
|AiHandling                   |string                           |
|AntiRollBarBiasFront         |double                           |
|AntiRollBarForce             |double                           |
|BrakeBiasFront               |double                           |
|BrakeForce                   |double                           |
|CamberStiffnesss             |double                           |
|CentreOfMassOffset           |struct<X:double,Y:double,Z:double>|
|ClutchChangeRateScaleDownShift|double                           |
|ClutchChangeRateScaleUpShift |double                           |
|CollisionDamageMult          |double                           |
|DeformationDamageMult        |double                           |
|DriveBiasFront               |double                           |
|DriveInertia                 |double                           |
|EngineDamageMult             |double                           |
|HandBrakeForce               |double                           |
|Id                           |string                           |
|InertiaMultiplier            |struct<X:double,Y:double,Z:double>|
|InitialDragCoeff             |double                           |
|InitialDriveForce            |double                           |
|InitialDriveGears            |bigint                           |
|InitialDriveMaxFlatVel       |double                           |
|LowSpeedTractionLossMult     |double                           |
|Mass                         |double                           |
|MonetaryValue                |bigint                           |
|OilVolume                    |double                           |
|PercentSubmerged             |double                           |
|PetrolTankVolume             |double                           |
|RollCentreHeightFront        |double                           |
|RollCentreHeightRear         |double                           |
|SeatOffsetDistX              |double                           |
|SeatOffsetDistY              |double                           |
|SeatOffsetDistZ              |double                           |
|SteeringLock                 |double                           |
|SuspensionBiasFront          |double                           |
|SuspensionCompDamp           |double                           |
```

```
|SuspensionForce               |double                           |
|SuspensionLowerLimit          |double                           |
|SuspensionRaise               |double                           |
|SuspensionReboundDamp         |double                           |
|SuspensionUpperLimit          |double                           |
|TractionBiasFront             |double                           |
|TractionCurveLateral          |double                           |
|TractionCurveMax              |double                           |
|TractionCurveMin              |double                           |
|TractionLossMult              |double                           |
|TractionSpringDeltaMax        |double                           |
|VehicleModels                 |array<string>                    |
|WeaponDamageMult              |double                           |
+------------------------------+---------------------------------+
```

```
veh_raw.select(
    "Name", F.col("DisplayName.English").alias("DisplayName_EN"), "Manufacturer", "Class", "Type",
    "DlcName", "HandlingId", "Price", "MonetaryValue", "MaxSpeed", "Acceleration", "MaxBraking"
).show(5, truncate=False)
```

```
+------+--------------+------------+----------+----+--------------+----------+-----+-------------+---
------+------------+----------+
|Name  |DisplayName_EN|Manufacturer|Class     |Type|DlcName
|HandlingId|Price|MonetaryValue|MaxSpeed |Acceleration|MaxBraking|
+------+--------------+------------+----------+----+--------------+----------+-----+-------------+---
------+------------+----------+
|adder |Adder         |TRUFFADE    |SUPER     |CAR |TitleUpdate    |ADDER     |-1   |80000
|51.77096 |0.32        |1.0       |
|Airbus|Airport Bus   |            |SERVICE   |CAR |TitleUpdate    |AIRBUS    |-1   |25000
|25.580894|0.12        |0.25      |
|Airtug|Airtug        |            |UTILITY   |CAR |TitleUpdate    |AIRTUG    |-1   |15000
|10.941742|0.06        |0.3       |
|akula |Akula         |            |HELICOPTER|HELI|mpchristmas2017|AKULA     |100  |52000
|61.089123|5.88        |3.5920403 |
|akuma |Akuma         |DINKA       |MOTORCYCLE|BIKE|TitleUpdate    |AKUMA     |-1   |20000
|48.333336|0.4         |1.2       |
+------+--------------+------------+----------+----+--------------+----------+-----+-------------+---
------+------------+----------+
only showing top 5 rows
```

Observações:

- Os tipos numéricos foram inferidos corretamente (`bigint` / `double`) — **não há problema de tipo** a corrigir na leitura.
- Há muitas colunas **aninhadas**: `DisplayName` e `ManufacturerDisplayName` (struct com o nome em 15 idiomas), `DimensionsMin/Max` (struct X/Y/Z), `Flags`, `Bones`, `DefaultColors`, `Extras` (arrays). Elas precisarão ser **achatadas** e só o que é relevante será mantido.
- Diversas colunas são técnicas do motor do jogo (`Hash`, `SignedHash`, `HexHash`, `Bones`, `LayoutId`, `DefaultHorn` ...) e não têm relação com as perguntas analíticas.

4.2 Valores ausentes (nulos e strings vazias)

```
print("== vehicles.json ==")
tabela(perfil_ausentes(veh_raw), ["coluna", "ausentes", "%"])
print("Nome em inglês (DisplayName.English) nulo:",
veh_raw.filter(F.col("DisplayName.English").isNull()).count())
```

```
== vehicles.json ==
+-------------+--------+----+
|coluna       |ausentes|%   |
+-------------+--------+----+
|DashboardType|67      |8.2 |
|DimensionsMax|5       |0.6 |
|DimensionsMin|5       |0.6 |
|Manufacturer |149     |18.3|
|Rewards      |793     |97.3|
+-------------+--------+----+
```

```
Nome em inglês (DisplayName.English) nulo: 4
```

```
veh_raw.filter(F.trim("Manufacturer") == "").groupBy("Type",
"Class").count().orderBy(F.desc("count")).show(10)
veh_raw.filter(F.col("DisplayName.English").isNull()).select("Name", "Type",
"DisplayName.Name").show()
```

```
+-------+----------+-----+
|   Type|     Class|count|
+-------+----------+-----+
|TRAILER|   UTILITY|   24|
|   HELI|HELICOPTER|   21|
|  PLANE|     PLANE|   19|
|    CAR| EMERGENCY|   16|
|    CAR|   UTILITY|   14|
|BICYCLE|     CYCLE|    9|
|    CAR|   SERVICE|    9|
|  TRAIN|      RAIL|    9|
|    CAR|  MILITARY|    8|
|    CAR|  OFF_ROAD|    4|
+-------+----------+-----+
only showing top 10 rows
+--------------+-------+--------------+
|          Name|   Type|          Name|
+--------------+-------+--------------+
|   docktrailer|TRAILER|   DOCKTRAILER|
|freighttrailer|TRAILER|freighttrailer|
|   proptrailer|TRAILER|   PROPTRAILER|
|       tanker2|TRAILER|        tanker|
+--------------+-------+--------------+
```

```
print("== vehicleHandlings.json ==")
tabela(perfil_ausentes(hdl_raw), ["coluna", "ausentes", "%"])
```

```
== vehicleHandlings.json ==
(nenhum registro)
```

Observações:

- `Manufacturer` vem como **string vazia** (não nula) em 149 veículos — uma ausência "disfarçada", que uma contagem simples de nulos não detectaria. São majoritariamente veículos utilitários, aeronaves, emergência, trens e bicicletas: veículos **genéricos, sem marca no jogo**.
- 4 veículos (todos *trailers*) não têm nome de exibição em inglês.
- `Rewards`, `TranslatedDescription`, `SubHandlingData`, `AiHandling` etc. são quase sempre nulos, mas não são relevantes para o problema e serão descartados.

4.3 Valores sentinela, inválidos ou impossíveis

Ausências também podem estar codificadas como números (ex.: `-1`, `0`, `100`).

```
print("Distribuição de Price:")
veh_raw.groupBy("Price").count().orderBy(F.desc("count")).show(6)
```

```
Distribuição de Price:
+------+-----+
| Price|count|
+------+-----+
|   100|  493|
|    -1|  314|
| 82000|    1|
|448000|    1|
|  9464|    1|
| 40000|    1|
```

```
+------+-----+
only showing top 6 rows
```

```
metricas = ["MaxSpeed", "Acceleration", "MaxBraking", "MaxBrakingMods", "MaxTraction", "Agility",
"MonetaryValue"]
tabela(
    [(m, veh_raw.filter(F.col(m) < 0).count(), veh_raw.filter(F.col(m) == 0).count()) for m in
metricas],
    ["coluna", "negativos", "zeros"],
)
```

```
+--------------+---------+-----+
|coluna        |negativos|zeros|
+--------------+---------+-----+
|MaxSpeed      |30       |0    |
|Acceleration  |5        |25   |
|MaxBraking    |5        |0    |
|MaxBrakingMods|5        |0    |
|MaxTraction   |5        |26   |
|Agility       |5        |0    |
|MonetaryValue |0        |5    |
+--------------+---------+-----+
```

```
print("Veículos com métricas negativas (por tipo):")
veh_raw.filter(F.col("MaxSpeed") < 0).groupBy("Type").count().show()

print("Carros (não-trailers) com métricas negativas:")
(veh_raw.filter((F.col("MaxSpeed") < 0) & (F.col("Type") != "TRAILER"))
    .select("Name", "Type", "DlcName", "MaxSpeed", "Acceleration", "MaxBraking", "Agility",
"MonetaryValue")
    .show(truncate=False))
```

```
Veículos com métricas negativas (por tipo):
+-------+-----+
|   Type|count|
+-------+-----+
|    CAR|    5|
|TRAILER|   25|
+-------+-----+

Carros (não-trailers) com métricas negativas:
+---------+----+-------+--------+------------+----------+-------+-------------+
|Name     |Type|DlcName|MaxSpeed|Acceleration|MaxBraking|Agility|MonetaryValue|
+---------+----+-------+--------+------------+----------+-------+-------------+
|arbitergt|CAR |mpg9ec |-1.0    |-1.0        |-1.0      |-1.0   |0            |
|astron2  |CAR |mpg9ec |-1.0    |-1.0        |-1.0      |-1.0   |0            |
|cyclone2 |CAR |mpg9ec |-1.0    |-1.0        |-1.0      |-1.0   |0            |
|ignus2   |CAR |mpg9ec |-1.0    |-1.0        |-1.0      |-1.0   |0            |
|s95      |CAR |mpg9ec |-1.0    |-1.0        |-1.0      |-1.0   |0            |
+---------+----+-------+--------+------------+----------+-------+-------------+
```

Observações:

- `Price` **não é um preço real:** 493 veículos têm `Price = 100` e 314 têm `Price = -1` (≈ 99% do total); só 8 veículos têm um valor diferente. A coluna é, na prática, um *placeholder* do jogo. O valor econômico real do veículo está em `MonetaryValue`.
- `-1` **é um valor sentinela** em todas as métricas de desempenho: velocidade, aceleração e frenagem negativas são fisicamente impossíveis.
  - 25 casos são **trailers** (reboques não têm motor próprio, logo não têm desempenho).
  - 5 casos são carros do DLC `mpg9ec` (versões exclusivas da edição *Expanded & Enhanced* para PS5/Xbox Series) — o extrator não obteve as estatísticas deles; os mesmos 5 têm `MonetaryValue = 0`.
- `MaxTraction = 0` em 26 veículos não-trailer é **legítimo** (aeronaves/barcos não dependem de tração no solo), assim como `MaxKnots` / `MoveResistance` = 0 fora de barcos/aeronaves.

4.4 Registros duplicados

```
total = veh_raw.count()
print("vehicles: linhas =", total,
      "| Name distintos =", veh_raw.select("Name").distinct().count(),
      "| lower(Name) distintos =", veh_raw.select(F.lower("Name")).distinct().count())

print("\nNomes de exibição repetidos (variantes do mesmo modelo):")
(veh_raw.groupBy(F.col("DisplayName.English").alias("display_name"))
    .agg(F.count("*").alias("qtd"), F.collect_list("Name").alias("modelos"))
    .filter("qtd > 1").orderBy(F.desc("qtd")).show(8, truncate=False))
```

```
vehicles: linhas = 815 | Name distintos = 815 | lower(Name) distintos = 815

Nomes de exibição repetidos (variantes do mesmo modelo):
+-------------+---+----------------------------------------------------------------------------------------
----------------------------------------+
|display_name |qtd|modelos
|
+-------------+---+----------------------------------------------------------------------------------------
----------------------------------------+
|Trailer      |13 |[raketrailer, tanker, tr2, tr3, tr4, trailerlogs, trailers, trailers2, trailers3,
trailers4, trailersmall, trflat, tvtrailer]|
|Freight Train|8  |[freight, freightcar, freightcar2, freightcont1, freightcont2, freightgrain,
metrotrain, tankercar]                      |
|NULL         |4  |[docktrailer, freighttrailer, proptrailer, tanker2]
|
|Avenger      |4  |[avenger, avenger2, avenger3, avenger4]
|
|Boxville     |4  |[boxville, boxville2, boxville3, boxville4]
|
|Burrito      |4  |[Burrito, burrito3, Burrito4, burrito5]
|
|Cargobob     |4  |[Cargobob, cargobob2, Cargobob3, Cargobob4]
|
|Dinghy       |4  |[Dinghy, dinghy2, dinghy3, dinghy4]
|
+-------------+---+----------------------------------------------------------------------------------------
----------------------------------------+
only showing top 8 rows
```

```
hdl_dup = hdl_raw.groupBy(F.upper("Id").alias("id")).count().filter("count > 1")
hdl_sem_tecnica = hdl_raw.drop("_ordem_origem")
conflitantes = (hdl_sem_tecnica.distinct()
                .groupBy(F.upper("Id").alias("id")).count().filter("count > 1"))

print("handlings: linhas =", hdl_raw.count(),
      "| Id distintos =", hdl_raw.select(F.upper("Id")).distinct().count())
print("Ids repetidos:", hdl_dup.count())
print("Linhas que são cópia exata de outra:", hdl_raw.count() - hdl_sem_tecnica.distinct().count())
print("Ids repetidos com valores DIFERENTES entre as cópias:", conflitantes.count())
conflitantes.show(truncate=False)
```

```
handlings: linhas = 747 | Id distintos = 723
Ids repetidos: 24
Linhas que são cópia exata de outra: 19
Ids repetidos com valores DIFERENTES entre as cópias: 5
+----------+-----+
|id        |count|
+----------+-----+
|STALION2  |2    |
|INSURGENT |2    |
|INNOVAT   |2    |
|STALION   |2    |
|INSURGENT2|2    |
+----------+-----+
```

Observações:

- Em `vehicles` **não há duplicatas**: `Name` é único (inclusive ignorando maiúsculas/minúsculas). Os 48 nomes de exibição repetidos são **variantes legítimas** do mesmo modelo (ex.: `boxville`, `boxville2` … são versões diferentes do mesmo furgão, com atributos distintos) — **não serão removidos**; a chave do conjunto será `Name`.
- Em `vehicleHandlings` **há duplicatas**: 24 `Id` aparecem duas vezes. 19 são **cópias exatas**; 5 têm pequenas diferenças em parâmetros de suspensão/tração (o mesmo *handling* foi redefinido por um DLC posterior). Isso, se não tratado, **duplicaria veículos no join**.

4.5 Formatos inconsistentes

```
print("Name com letras maiúsculas:", veh_raw.filter(F.col("Name") != F.lower("Name")).count())
veh_raw.filter(F.col("Name") != F.lower("Name")).select("Name").show(8)
```

```
Name com letras maiúsculas: 94
+---------+
|     Name|
+---------+
|   Airbus|
|   Airtug|
|AMBULANCE|
|   Baller|
| BARRACKS|
|BARRACKS2|
|BARRACKS3|
|   Benson|
+---------+
only showing top 8 rows
```

```
ids_exatos = hdl_raw.select(F.col("Id").alias("hid")).distinct()
ids_upper = hdl_raw.select(F.upper(F.trim("Id")).alias("hid")).distinct()

sem_match_exato = veh_raw.join(ids_exatos, F.col("HandlingId") == F.col("hid"), "left_anti")
sem_match_upper = veh_raw.join(ids_upper, F.upper(F.trim("HandlingId")) == F.col("hid"),
"left_anti")
print("Veículos sem handling (comparação exata)        :", sem_match_exato.count())
print("Veículos sem handling (após padronizar caixa)   :", sem_match_upper.count())

(sem_match_exato.select("Name", "HandlingId").limit(5)
    .join(hdl_raw.select("Id"), F.upper("HandlingId") == F.upper("Id")).show())
```

```
Veículos sem handling (comparação exata)        : 78
Veículos sem handling (após padronizar caixa)   : 0
+------+----------+------+
|  Name|HandlingId|    Id|
+------+----------+------+
|Benson|    BENSON|benson|
| caddy|     CADDY| caddy|
| bifta|     BIFTA| bifta|
| alpha|     ALPHA| alpha|
| blade|     BLADE| blade|
+------+----------+------+
```

```
print("Mesmo fabricante escrito com códigos diferentes:")
(veh_raw.filter(F.trim("Manufacturer") != "")
    .groupBy(F.col("ManufacturerDisplayName.English").alias("fabricante"))
    .agg(F.collect_set("Manufacturer").alias("codigos"), F.count("*").alias("veiculos"))
    .filter(F.size("codigos") > 1).show(truncate=False))
```

```
Mesmo fabricante escrito com códigos diferentes:
+----------+--------------------+--------+
|fabricante|codigos             |veiculos|
+----------+--------------------+--------+
|Benefactor|[BENEFACTOR, BENEFAC]|29      |
|Lampadati |[LAMPADATI, LAMPADA] |15      |
|Ubermacht |[UBERMACH, UBERMACHT]|14      |
+----------+--------------------+--------+
```

Observações:

- **Caixa inconsistente em chaves:** 94 valores de `Name` têm letras maiúsculas (`Baller`, `BARRACKS`, `Burrito`...) enquanto a maioria é minúscula. O mesmo ocorre entre `HandlingId` e `Id`: **78 veículos não encontram seu *handling*** numa comparação exata (ex.: `ALPHA` × `Alpha`) — e **todos** passam a casar após padronizar para maiúsculas.
- **Códigos de fabricante truncados:** `BENEFAC` / `BENEFACTOR`, `LAMPADA` / `LAMPADATI` e `UBERMACH` / `UBERMACHT` representam o mesmo fabricante. Agrupar por `Manufacturer` dividiria esses fabricantes em dois. O nome de exibição (`ManufacturerDisplayName.English`) é consistente e será usado.

4.6 Consistência das categorias

```
veh_raw.groupBy("Type").count().orderBy(F.desc("count")).show(20)
veh_raw.groupBy("Class", "ClassId").count().orderBy("ClassId").show(30)
```

```
+--------------------+-----+
|                Type|count|
+--------------------+-----+
|                 CAR|  602|
|                BIKE|   56|
|               PLANE|   41|
|                HELI|   30|
|             TRAILER|   25|
|                BOAT|   22|
|               TRAIN|    9|
|             BICYCLE|    9|
|            QUADBIKE|    8|
|           SUBMARINE|    4|
|AMPHIBIOUS_AUTOMO...|    3|
|               BLIMP|    3|
|        SUBMARINECAR|    2|
| AMPHIBIOUS_QUADBIKE|    1|
+--------------------+-----+
+-------------+-------+-----+
|        Class|ClassId|count|
+-------------+-------+-----+
|      COMPACT|      0|   18|
|        SEDAN|      1|   38|
|          SUV|      2|   41|
|        COUPE|      3|   17|
|       MUSCLE|      4|   86|
|SPORT_CLASSIC|      5|   45|
|        SPORT|      6|  105|
|        SUPER|      7|   57|
|   MOTORCYCLE|      8|   58|
|     OFF_ROAD|      9|   65|
|   INDUSTRIAL|     10|   11|
|      UTILITY|     11|   44|
|          VAN|     12|   39|
|        CYCLE|     13|    9|
|         BOAT|     14|   26|
|   HELICOPTER|     15|   29|
|        PLANE|     16|   44|
|      SERVICE|     17|   13|
|    EMERGENCY|     18|   19|
|     MILITARY|     19|   17|
|   COMMERCIAL|     20|   21|
|         RAIL|     21|    9|
|   OPEN_WHEEL|     22|    4|
+-------------+-------+-----+
```

```
for c in ["Type", "Class", "DlcName"]:
    fora_padrao = veh_raw.filter(F.col(c) != F.trim(F.col(c))).count()
    print(f"{c:<8}: {veh_raw.select(c).distinct().count():>2} categorias | com espaços extras:
{fora_padrao}")
print("Class x ClassId 1:1 ->", veh_raw.select("Class", "ClassId").distinct().count() ==
veh_raw.select("Class").distinct().count())
```

```
Type    : 14 categorias | com espaços extras: 0
Class   : 23 categorias | com espaços extras: 0
DlcName : 42 categorias | com espaços extras: 0
Class x ClassId 1:1 -> True
```

Observações: as categorias de `Type`, `Class` e `DlcName` são **consistentes** (sem variações de grafia, espaços extras ou caixa misturada) e `Class` ↔ `ClassId` é 1:1. **Não há tratamento necessário** aqui — a única exceção é a categoria `TRAILER`, discutida na seção 5.

4.7 Possíveis outliers

Usamos o critério do **IQR** (valores fora de `[Q1 − 1,5·IQR ; Q3 + 1,5·IQR]`), primeiro no conjunto inteiro e depois **dentro de cada `Type`** — porque comparar a velocidade de um avião com a de uma bicicleta não faz sentido. Os sentinelas (`< 0`) são excluídos do cálculo.

```
METRICAS_OUTLIER = ["MaxSpeed", "Acceleration", "MonetaryValue"]
validos = veh_raw.filter((F.col("MaxSpeed") > 0) & (F.col("MonetaryValue") > 0))


def contar_outliers(df, col, por=None):
    chaves = [por] if por else []
    q = df.groupBy(*chaves).agg(F.percentile_approx(col, [0.25, 0.75]).alias("q"))
    q = q.select(*chaves, F.col("q")[0].alias("q1"), F.col("q")[1].alias("q3"))
    j = df.crossJoin(q) if not por else df.join(q, por)
    iqr = F.col("q3") - F.col("q1")
    return j.filter((F.col(col) < F.col("q1") - 1.5 * iqr) | (F.col(col) > F.col("q3") + 1.5 * iqr))


tabela(
    [(m, contar_outliers(validos, m).count(), contar_outliers(validos, m, "Type").count()) for m in
METRICAS_OUTLIER],
    ["metrica", "outliers_global", "outliers_dentro_do_Type"],
)
```

```
+-------------+---------------+-----------------------+
|metrica      |outliers_global|outliers_dentro_do_Type|
+-------------+---------------+-----------------------+
|MaxSpeed     |65             |20                     |
|Acceleration |107            |22                     |
|MonetaryValue|151            |112                    |
+-------------+---------------+-----------------------+
```

```
print("Resumo por tipo (MaxSpeed em m/s):")
(validos.groupBy("Type")
    .agg(F.count("*").alias("n"),
         F.round(F.min("MaxSpeed"), 1).alias("min_speed"),
         F.round(F.max("MaxSpeed"), 1).alias("max_speed"),
         F.round(F.max("Acceleration"), 2).alias("max_accel"),
         F.max("MonetaryValue").alias("max_valor"))
    .orderBy(F.desc("max_speed")).show(20))

print("Maiores outliers de MaxSpeed dentro de CAR:")
(contar_outliers(validos, "MaxSpeed", "Type").filter("Type = 'CAR'")
    .select("Name", "Class", "MaxSpeed", "Acceleration",
"MonetaryValue").orderBy(F.desc("MaxSpeed")).show(10))
```

```
Resumo por tipo (MaxSpeed em m/s):
+--------------------+---+---------+---------+---------+---------+
|                Type|  n|min_speed|max_speed|max_accel|max_valor|
+--------------------+---+---------+---------+---------+---------+
|               PLANE| 41|     30.2|    109.8|    21.56|    45000|
|               BLIMP|  3|     70.0|     70.0|     6.08|    52000|
|                HELI| 30|     47.3|     69.5|     6.86|    52000|
|                 CAR|597|      5.0|     53.9|     0.76|   500000|
|                BIKE| 56|     23.5|     52.1|     0.82|    20000|
|        SUBMARINECAR|  2|     45.9|     47.0|      0.3|    35000|
```

```
|               BOAT| 22|      6.0|     46.7|     18.0|    40000|
|            QUADBIKE|  8|     24.7|     42.2|     0.38|    12000|
| AMPHIBIOUS_QUADBIKE|  1|     40.9|     40.9|     0.27|    12000|
|AMPHIBIOUS_AUTOMO...|  3|     23.0|     39.6|     0.25|    50000|
|               TRAIN|  9|     26.7|     26.7|      0.2|   110000|
|           SUBMARINE|  4|     25.0|     25.0|     10.5|    40000|
|             BICYCLE|  9|     10.8|     21.0|     0.17|    12000|
+--------------------+---+---------+---------+---------+---------+

Maiores outliers de MaxSpeed dentro de CAR:
+--------+----------+---------+------------+-------------+
|    Name|     Class| MaxSpeed|Acceleration|MonetaryValue|
+--------+----------+---------+------------+-------------+
|   RHINO|  MILITARY|18.333336|        0.11|        40000|
|khanjali|  MILITARY|18.122084|         0.1|        40000|
|tractor3|   UTILITY| 14.45938|        0.15|        50000|
|tractor2|   UTILITY| 14.45938|        0.15|        50000|
|    dump|INDUSTRIAL|14.333334|        0.19|        40000|
| TRACTOR|   UTILITY|13.333335|        0.08|        50000|
|  cutter|INDUSTRIAL|12.761157|        0.16|        25000|
|  Airtug|   UTILITY|10.941742|        0.06|        15000|
|FORKLIFT|   UTILITY|10.000001|        0.18|        25000|
|minitank|  MILITARY| 9.729744|        0.06|        40000|
+--------+----------+---------+------------+-------------+
only showing top 10 rows
```

Observações: a maior parte dos "outliers globais" desaparece quando a análise é feita **por tipo**: aviões chegam a ~110 m/s e têm aceleração 20× maior que a dos carros por serem outra categoria, não por erro. Os outliers que restam dentro de cada tipo (ex.: carros de corrida/*open wheel* muito rápidos, veículos militares muito caros) são **valores plausíveis do jogo**, não erros de medição.

4.8 Integridade referencial entre as fontes

```python
handling_usados = veh_raw.select(F.upper(F.trim("HandlingId")).alias("hid")).distinct()
orfaos = ids_upper.join(handling_usados, "hid", "left_anti")
print("Handlings não usados por nenhum veículo:", orfaos.count())
orfaos.show(truncate=False)

compartilhados = veh_raw.groupBy(F.upper("HandlingId").alias("hid")).count().filter("count > 1")
print("Handlings compartilhados por mais de um veículo:", compartilhados.count())
```

```
Handlings não usados por nenhum veículo: 12
+--------+
|hid     |
+--------+
|RHAPSODY|
|TRFLAT  |
|SPSMALL |
|TRAILER2|
|GBURRITO|
|URANUS  |
|TRASH2  |
|BLIMP3  |
|SERRANO2|
|SPMED   |
|MARBELLA|
|FBIOLD  |
+--------+

Handlings compartilhados por mais de um veículo: 76
```

Observações: 12 *handlings* não pertencem a nenhum veículo do catálogo (sobras de versões antigas/veículos removidos) — serão naturalmente descartados por um *left join* a partir dos veículos. O compartilhamento de *handling* entre veículos (76 casos) é **esperado** (variantes usam a mesma física) e confirma a relação N:1.

4.9 Resumo dos problemas de qualidade

#	Problema	Onde	Volume	Relevância
Q1	Ausência disfarçada (string vazia)	`vehicles.Manufacturer`	149 (18%)	Alta – afeta P4
Q2	Nome de exibição ausente	`DisplayName.English`	4	Baixa
Q3	Coluna sem informação real (placeholder `100` / `-1`)	`vehicles.Price`	807 (99%)	Alta – invalidaria P2 se usada
Q4	Sentinela `-1` em métricas de desempenho / `0` em valor	`MaxSpeed`, `Acceleration`, `MaxBraking`, `Agility`, `MonetaryValue`	30 (25 trailers + 5 carros `mpg9ec`)	Alta – distorce médias de P1–P5
Q5	Duplicatas (exatas e conflitantes)	`vehicleHandlings.Id`	24 Ids	Alta – duplicaria linhas no join
Q6	Caixa inconsistente em chaves	`Name`, `HandlingId` × `Id`	94 / 78 joins perdidos	Alta – perderia 10% dos veículos no join
Q7	Códigos de categoria truncados	`vehicles.Manufacturer`	3 fabricantes	Média – afeta P4
Q8	Estruturas aninhadas e colunas técnicas	ambos	~40 colunas	Média – dificulta uso
Q9	Outliers	métricas	explicados pelo `Type`	Baixa – não são erros
Q10	Registros fora do escopo (sem motor)	`Type = TRAILER`	25	Média
–	Tipos de dados / categorias `Type` / `Class`	ambos	sem problemas	–

RA1 / 05

Limpeza e preparação dos dados

Cada passo abaixo referencia o problema (Q#) da seção 4.9 e a justificativa da decisão.

5.1 `vehicleHandlings`: padronizar chave e remover duplicatas (Q5, Q6, Q8)

- **Chave padronizada** para maiúsculas sem espaços (`handling_id`), permitindo o join (Q6).
- **Deduplicação:** para cada `handling_id`, mantemos a **última ocorrência no arquivo** (`_ordem_origem` mais alto). Para as 19 cópias exatas a escolha é indiferente; para as 5 conflitantes, assumimos que a definição posterior corresponde à redefinição feita por um DLC mais recente, que é a que vale no jogo. As diferenças são pequenas (parâmetros de suspensão/tração) e não afetam as variáveis principais.
- **Seleção de colunas:** mantemos apenas os atributos físicos ligados à pergunta P3, com nomes em *snake_case* e unidade explícita quando conhecida. `MonetaryValue` do *handling* é descartado (o valor oficial é o do veículo).

```
ultima_ocorrencia = Window.partitionBy("handling_id").orderBy(F.desc("_ordem_origem"))

hdl = (
    hdl_raw
    .withColumn("handling_id", F.upper(F.trim("Id")))
    .withColumn("_rn", F.row_number().over(ultima_ocorrencia))
    .filter("_rn = 1")
    .select(
        "handling_id",
        F.col("Mass").alias("mass_kg"),
        F.col("InitialDragCoeff").alias("drag_coeff"),
        F.col("DriveBiasFront").alias("drive_bias_front"),
        F.col("InitialDriveGears").cast("int").alias("drive_gears"),
        F.col("InitialDriveForce").alias("drive_force"),
        F.col("InitialDriveMaxFlatVel").alias("drive_max_flat_vel"),
        F.col("BrakeForce").alias("brake_force"),
        F.col("HandBrakeForce").alias("handbrake_force"),
        F.col("SteeringLock").alias("steering_lock_deg"),
        F.col("TractionCurveMax").alias("traction_curve_max"),
        F.col("TractionCurveMin").alias("traction_curve_min"),
        F.col("SuspensionForce").alias("suspension_force"),
        F.col("CollisionDamageMult").alias("collision_damage_mult"),
        F.col("DeformationDamageMult").alias("deformation_damage_mult"),
        F.col("EngineDamageMult").alias("engine_damage_mult"),
        F.col("PetrolTankVolume").alias("petrol_tank_volume"),
    )
)
print("handlings: antes =", hdl_raw.count(), "| depois =", hdl.count(),
      "| handling_id único:", hdl.count() == hdl.select("handling_id").distinct().count())
```

```
handlings: antes = 747 | depois = 723 | handling_id único: True
```

5.2 `vehicles`: escopo, achatamento, padronização e sentinelas

Decisão	Problema	Justificativa
Remover `Type = TRAILER`	Q10, Q4	Reboques não têm propulsão própria: todas as métricas de desempenho são `-1`. Não respondem a nenhuma pergunta sobre desempenho e distorceriam as estatísticas.
`name` em minúsculas; `handling_id` em maiúsculas	Q6	Chaves com caixa única; garante o join e evita duplicidade lógica.

Decisão	Problema	Justificativa
`display_name` = `DisplayName.English`; se nulo, usar `Name`	Q2, Q8	Achatar o struct multilíngue mantendo só o idioma de análise; *fallback* evita nulos (após remover trailers, não deve sobrar nenhum).
`manufacturer` = `ManufacturerDisplayName.English`; vazio → `"Sem fabricante"`	Q1, Q7	O nome de exibição unifica códigos truncados (`BENEFAC` / `BENEFACTOR`). A ausência não é desconhecimento: são veículos **genéricos sem marca** no jogo, então uma categoria explícita é mais fiel que nulo e evita perder linhas em agrupamentos.
Remover `Price`	Q3	99% *placeholder*; manter a coluna induziria análises erradas. `monetary_value` é a métrica de valor.
Métricas negativas → `null`; `MonetaryValue = 0` → `null`	Q4	`-1` e `0` são sentinelas de "dado não extraído". Imputar (média/mediana) inventaria desempenho para 5 carros específicos; removê-los perderia veículos válidos do catálogo. Mantemos a linha com nulo e uma *flag* `has_performance_stats`.
`max_speed_kmh` = `MaxSpeed` × 3,6	padronização	`MaxSpeed` está em m/s (unidade interna do jogo); km/h facilita a interpretação. A coluna original em m/s é mantida.
Dimensões → `length_m`, `width_m`, `height_m`	Q8	Achatar os structs X/Y/Z em medidas diretas (max − min).
Descartar colunas técnicas (`Hash*`, `Bones`, `DefaultColors`, `Extras`, `ModKits`, `LayoutId`, `DefaultHorn`, `MaxKnots`, `MoveResistance` ...)	Q8	Não se relacionam ao problema; `MaxKnots` / `MoveResistance` só têm valor para barcos/aeronaves.
Manter outliers	Q9	São explicados pelo tipo de veículo e plausíveis no jogo; removê-los eliminaria justamente os veículos de alto desempenho que interessam a P1/P2. Nos próximos RAs as comparações devem ser feitas **dentro de cada `vehicle_type`**.

```
def nao_negativo(col):
    # Sentinela: valores < 0 são impossíveis para estas métricas -> null.
    return F.when(F.col(col) >= 0, F.col(col))


veh = (
    veh_raw
    .filter(F.col("Type") != "TRAILER")
    .select(
        F.lower(F.trim("Name")).alias("name"),
        F.coalesce(F.col("DisplayName.English"), F.col("Name")).alias("display_name"),
        F.coalesce(
            F.when(F.trim("Manufacturer") != "", F.col("ManufacturerDisplayName.English")),
            F.lit("Sem fabricante"),
        ).alias("manufacturer"),
        F.col("Type").alias("vehicle_type"),
        F.col("Class").alias("vehicle_class"),
        F.col("ClassId").cast("int").alias("class_id"),
        F.col("DlcName").alias("dlc_name"),
        F.upper(F.trim("HandlingId")).alias("handling_id"),
        F.col("Seats").cast("int").alias("seats"),
        F.col("WheelsCount").cast("int").alias("wheels_count"),
        F.when(F.col("MonetaryValue") > 0,
F.col("MonetaryValue")).cast("long").alias("monetary_value"),
        nao_negativo("MaxSpeed").alias("max_speed_ms"),
```

```
        F.round(nao_negativo("MaxSpeed") * 3.6, 2).alias("max_speed_kmh"),
        nao_negativo("Acceleration").alias("acceleration"),
        nao_negativo("MaxBraking").alias("max_braking"),
        nao_negativo("MaxBrakingMods").alias("max_braking_mods"),
        nao_negativo("MaxTraction").alias("max_traction"),
        nao_negativo("Agility").alias("agility"),
        (F.col("MaxSpeed") >= 0).alias("has_performance_stats"),
        F.col("DefaultBodyHealth").alias("default_body_health"),
        F.col("SpawnFrequency").alias("spawn_frequency"),
        F.round(F.col("DimensionsMax.Y") - F.col("DimensionsMin.Y"), 3).alias("length_m"),
        F.round(F.col("DimensionsMax.X") - F.col("DimensionsMin.X"), 3).alias("width_m"),
        F.round(F.col("DimensionsMax.Z") - F.col("DimensionsMin.Z"), 3).alias("height_m"),
        F.col("HasConvertibleRoof").alias("has_convertible_roof"),
        F.col("HasSirens").alias("has_sirens"),
        F.col("HasArmoredWindows").alias("has_armored_windows"),
        F.col("HasParachute").alias("has_parachute"),
        F.col("HasKers").alias("has_kers"),
        (F.size("Weapons") > 0).alias("has_weapons"),
        F.coalesce("Flags", F.array().cast("array<string>")).alias("flags"),
    )
)
print("vehicles: antes =", veh_raw.count(), "| depois de remover trailers =", veh.count())
```

```
vehicles: antes = 815 | depois de remover trailers = 790
```

5.3 Integração `vehicles` + `vehicleHandlings`

Left join a partir dos veículos (o veículo é a unidade de análise; *handlings* órfãos ficam de fora). Com as chaves padronizadas e os *handlings* deduplicados, esperamos **nenhum veículo sem par** e **nenhuma linha duplicada**.

```
vehicles_clean = veh.join(hdl, "handling_id", "left")

sem_handling = vehicles_clean.filter(F.col("mass_kg").isNull()).count()
print("linhas antes do join:", veh.count(), "| depois:", vehicles_clean.count(), "| sem handling:",
sem_handling)

colunas_finais = ["name", "display_name", "manufacturer", "vehicle_type", "vehicle_class",
"class_id", "dlc_name",
                  "handling_id"] + [c for c in vehicles_clean.columns if c not in
                  ("name", "display_name", "manufacturer", "vehicle_type", "vehicle_class",
"class_id", "dlc_name", "handling_id")]
vehicles_clean = vehicles_clean.select(*colunas_finais).orderBy("name").cache()
vehicles_clean.printSchema()
```

```
linhas antes do join: 790 | depois: 790 | sem handling: 0
root
 |-- name: string (nullable = true)
 |-- display_name: string (nullable = true)
 |-- manufacturer: string (nullable = false)
 |-- vehicle_type: string (nullable = true)
 |-- vehicle_class: string (nullable = true)
 |-- class_id: integer (nullable = true)
 |-- dlc_name: string (nullable = true)
 |-- handling_id: string (nullable = true)
 |-- seats: integer (nullable = true)
 |-- wheels_count: integer (nullable = true)
 |-- monetary_value: long (nullable = true)
 |-- max_speed_ms: double (nullable = true)
 |-- max_speed_kmh: double (nullable = true)
 |-- acceleration: double (nullable = true)
 |-- max_braking: double (nullable = true)
 |-- max_braking_mods: double (nullable = true)
 |-- max_traction: double (nullable = true)
 |-- agility: double (nullable = true)
 |-- has_performance_stats: boolean (nullable = true)
 |-- default_body_health: double (nullable = true)
 |-- spawn_frequency: double (nullable = true)
 |-- length_m: double (nullable = true)
```

```
|-- width_m: double (nullable = true)
|-- height_m: double (nullable = true)
|-- has_convertible_roof: boolean (nullable = true)
|-- has_sirens: boolean (nullable = true)
|-- has_armored_windows: boolean (nullable = true)
|-- has_parachute: boolean (nullable = true)
|-- has_kers: boolean (nullable = true)
|-- has_weapons: boolean (nullable = false)
|-- flags: array (nullable = false)
|    |-- element: string (containsNull = true)
|-- mass_kg: double (nullable = true)
|-- drag_coeff: double (nullable = true)
|-- drive_bias_front: double (nullable = true)
|-- drive_gears: integer (nullable = true)
|-- drive_force: double (nullable = true)
|-- drive_max_flat_vel: double (nullable = true)
|-- brake_force: double (nullable = true)
|-- handbrake_force: double (nullable = true)
|-- steering_lock_deg: double (nullable = true)
|-- traction_curve_max: double (nullable = true)
|-- traction_curve_min: double (nullable = true)
|-- suspension_force: double (nullable = true)
|-- collision_damage_mult: double (nullable = true)
|-- deformation_damage_mult: double (nullable = true)
|-- engine_damage_mult: double (nullable = true)
|-- petrol_tank_volume: double (nullable = true)
```

RA1 / 06

Conjunto de dados resultante do RA1

6.1 Validações automáticas

Cada regra abaixo verifica que um problema da seção 4.9 foi de fato resolvido. Se alguma falhar, a célula interrompe a execução.

```
df = vehicles_clean
metricas_finais = ["max_speed_ms", "acceleration", "max_braking", "max_traction", "agility",
"monetary_value"]

checks = {
    "name é chave única (sem duplicatas)": df.count() == df.select("name").distinct().count(),
    "name sem nulos e em minúsculas (Q6)": df.filter(F.col("name").isNull() | (F.col("name") !=
F.lower("name"))).count() == 0,
    "join não duplicou linhas (Q5)": df.count() == veh.count(),
    "todo veículo tem handling (Q6)": df.filter(F.col("mass_kg").isNull()).count() == 0,
    "manufacturer sem nulos/vazios (Q1)": df.filter(F.col("manufacturer").isNull() |
(F.trim("manufacturer") == "")).count() == 0,
    "códigos truncados unificados (Q7)": df.filter(F.upper("manufacturer").isin("BENEFAC",
"LAMPADA", "UBERMACH")).count() == 0,
    "display_name sem nulos (Q2)": df.filter(F.col("display_name").isNull()).count() == 0,
    "sem sentinelas negativos/zero nas métricas (Q4)": df.filter(" OR ".join(f"{m} < 0" for m in
metricas_finais) + " OR monetary_value = 0").count() == 0,
    "nulos de desempenho só onde has_performance_stats = false (Q4)":
df.filter(F.col("max_speed_ms").isNull() & F.col("has_performance_stats")).count() == 0,
    "sem trailers (Q10)": df.filter(F.col("vehicle_type") == "TRAILER").count() == 0,
    "coluna Price removida (Q3)": "price" not in [c.lower() for c in df.columns],
}
for regra, ok in checks.items():
    print("OK    " if ok else "FALHOU", regra)
assert all(checks.values()), "Alguma validação falhou"
```

```
OK     name é chave única (sem duplicatas)
OK     name sem nulos e em minúsculas (Q6)
OK     join não duplicou linhas (Q5)
OK     todo veículo tem handling (Q6)
OK     manufacturer sem nulos/vazios (Q1)
OK     códigos truncados unificados (Q7)
OK     display_name sem nulos (Q2)
OK     sem sentinelas negativos/zero nas métricas (Q4)
OK     nulos de desempenho só onde has_performance_stats = false (Q4)
OK     sem trailers (Q10)
OK     coluna Price removida (Q3)
```

6.2 Comparativo de qualidade: antes × depois

```
antes = {
    "linhas (vehicles)": veh_raw.count(),
    "fabricante vazio": veh_raw.filter(F.trim("Manufacturer") == "").count(),
    "códigos de fabricante distintos": veh_raw.select("Manufacturer").distinct().count(),
    "métricas com sentinela (<0)": veh_raw.filter(F.col("MaxSpeed") < 0).count(),
    "valor monetário = 0": veh_raw.filter(F.col("MonetaryValue") == 0).count(),
    "veículos sem handling no join": sem_match_exato.count(),
    "handlings duplicados": hdl_raw.count() - hdl_raw.select(F.upper("Id")).distinct().count(),
    "colunas": len(veh_raw.columns) - 1,
}
depois = {
    "linhas (vehicles)": df.count(),
    "fabricante vazio": df.filter(F.col("manufacturer").isNull() | (F.trim("manufacturer") ==
"")).count(),
    "códigos de fabricante distintos": df.select("manufacturer").distinct().count(),
    "métricas com sentinela (<0)": df.filter(F.col("max_speed_ms") < 0).count(),
```

```
    "valor monetário = 0": df.filter(F.col("monetary_value") == 0).count(),
    "veículos sem handling no join": df.filter(F.col("mass_kg").isNull()).count(),
    "handlings duplicados": hdl.count() - hdl.select("handling_id").distinct().count(),
    "colunas": len(df.columns),
}
tabela([(k, int(antes[k]), int(depois[k])) for k in antes], ["indicador", "antes", "depois"])
print("Veículos mantidos com métricas nulas (sem estatísticas no jogo):",
      [r["name"] for r in df.filter(~F.col("has_performance_stats")).select("name").collect()])
```

```
+------------------------------+-----+------+
|indicador                     |antes|depois|
+------------------------------+-----+------+
|linhas (vehicles)             |815  |790   |
|fabricante vazio              |149  |0     |
|códigos de fabricante distintos|64   |60    |
|métricas com sentinela (<0)   |30   |0     |
|valor monetário = 0           |5    |0     |
|veículos sem handling no join |78   |0     |
|handlings duplicados          |24   |0     |
|colunas                       |53   |47    |
+------------------------------+-----+------+

Veículos mantidos com métricas nulas (sem estatísticas no jogo): ['arbitergt', 'astron2', 'cyclone2',
'ignus2', 's95']
```

6.3 Amostra e resumo do conjunto final

```
df.select("name", "display_name", "manufacturer", "vehicle_type", "vehicle_class", "dlc_name",
          "monetary_value", "max_speed_kmh", "acceleration", "max_braking", "mass_kg",
"drive_gears").show(10, truncate=False)
df.select("monetary_value", "max_speed_kmh", "acceleration", "max_braking", "mass_kg",
"drive_force").summary(
    "count", "mean", "min", "50%", "max").show()
tabela(perfil_ausentes(df), ["coluna", "ausentes", "%"])
```

```
+-----------+--------------+--------------+------------+-------------+---------------+--------------+-
------------+------------+-----------+-------+-----------+
|name       |display_name  |manufacturer  |vehicle_type|vehicle_class|dlc_name
|monetary_value|max_speed_kmh|acceleration|max_braking|mass_kg|drive_gears|
+-----------+--------------+--------------+------------+-------------+---------------+--------------+-
------------+------------+-----------+-------+-----------+
|adder      |Adder         |Truffade      |CAR         |SUPER        |TitleUpdate    |80000
|186.38       |0.32        |1.0        |1800.0 |6          |
|airbus     |Airport Bus   |Sem fabricante|CAR         |SERVICE      |TitleUpdate    |25000
|92.09        |0.12        |0.25       |9000.0 |4          |
|airtug     |Airtug        |Sem fabricante|CAR         |UTILITY      |TitleUpdate    |15000
|39.39        |0.06        |0.3        |1400.0 |1          |
|akula      |Akula         |Sem fabricante|HELI        |HELICOPTER   |mpchristmas2017|52000
|219.92       |5.88        |3.5920403  |8000.0 |1          |
|akuma      |Akuma         |Dinka         |BIKE        |MOTORCYCLE   |TitleUpdate    |20000
|174.0        |0.4         |1.2        |250.0  |5          |
|alkonost   |RO-86 Alkonost|Sem fabricante|PLANE       |PLANE        |mpheist4       |45000
|297.43       |13.524      |11.173427  |55500.0|1          |
|alpha      |Alpha         |Albany        |CAR         |SPORT        |mpbusiness     |80000
|178.34       |0.33        |1.0        |1800.0 |6          |
|alphaz1    |Alpha-Z1      |Buckingham    |PLANE       |PLANE        |mpsmuggler     |45000
|360.0        |20.58       |20.58      |450.0  |1          |
|ambulance  |Ambulance     |Sem fabricante|CAR         |EMERGENCY    |TitleUpdate    |40000
|143.68       |0.18        |0.6        |2500.0 |5          |
|annihilator|Annihilator   |Sem fabricante|HELI        |HELICOPTER   |TitleUpdate    |52000
|170.12       |4.606       |2.1766338  |13000.0|1          |
+-----------+--------------+--------------+------------+-------------+---------------+--------------+-
------------+------------+-----------+-------+-----------+
only showing top 10 rows
+-------+------------------+------------------+-----------------+------------------+----------------+-
-----------------+
|summary|    monetary_value|     max_speed_kmh|     acceleration|       max_braking|         mass_kg|
drive_force|
+-------+------------------+------------------+-----------------+------------------+----------------+-
-----------------+
```

```
|  count|               785|               785|               785|               785|               790|
790|
|
mean|58668.789808917194|157.71771974522284|1.420699373757961|1.3028514569426743|3565.63164556962|0.669
7325316455699|
|    min|             10000|              18.0|              0.05|               0.1|              85.0|
0.001|
|    50%|             40000|             158.4|             0.285|               0.8|            1800.0|
0.27|
|    max|            500000|            395.15|         21.560001|          22.41702|           60000.0|
18.0|
+-------+------------------+------------------+------------------+------------------+------------------+-
-----------------+
+----------------+--------+---+
|coluna          |ausentes|%  |
+----------------+--------+---+
|monetary_value  |5       |0.6|
|max_speed_ms    |5       |0.6|
|max_speed_kmh   |5       |0.6|
|acceleration    |5       |0.6|
|max_braking     |5       |0.6|
|max_braking_mods|5       |0.6|
|max_traction    |5       |0.6|
|agility         |5       |0.6|
|length_m        |5       |0.6|
|width_m         |5       |0.6|
|height_m        |5       |0.6|
+----------------+--------+---+
```

6.4 Dicionário de dados

Coluna	Tipo	Descrição
`name`	string	**Chave.** Nome do modelo do veículo no jogo (minúsculas)
`display_name`	string	Nome exibido ao jogador (inglês)
`manufacturer`	string	Fabricante (nome de exibição); `"Sem fabricante"` para veículos genéricos
`vehicle_type`	string	Tipo físico: CAR, BIKE, PLANE, HELI, BOAT...
`vehicle_class` / `class_id`	string / int	Classe do jogo: SUPER, SPORT, MUSCLE...
`dlc_name`	string	Pacote em que o veículo foi adicionado (`TitleUpdate` = jogo base)
`handling_id`	string	Chave do perfil de dirigibilidade (maiúsculas)
`seats`, `wheels_count`	int	Assentos e rodas
`monetary_value`	long	Valor monetário do veículo no jogo (US$ do jogo); nulo se não extraído
`max_speed_ms` / `max_speed_kmh`	double	Velocidade máxima estimada (m/s e km/h)
`acceleration`, `max_braking`, `max_braking_mods`, `max_traction`, `agility`	double	Métricas de desempenho (escala interna do jogo)
`has_performance_stats`	boolean	`false` quando o jogo não fornece as métricas (5 veículos `mpg9ec`)
`default_body_health`, `spawn_frequency`	double	Vida padrão da carroceria e frequência de aparição no trânsito
`length_m`, `width_m`, `height_m`	double	Dimensões da caixa delimitadora (m)

Coluna	Tipo	Descrição
`has_*`	boolean	Teto conversível, sirene, vidros blindados, paraquedas, KERS, armas
`flags`	array<string>	Marcadores do jogo (ex.: `FLAG_SPORTS`, `FLAG_RICH_CAR`)
`mass_kg`	double	Massa (kg) — *handling*
`drag_coeff`	double	Coeficiente de arrasto — *handling*
`drive_bias_front`	double	Distribuição da tração (0 = traseira, 1 = dianteira, 0,5 = 4x4)
`drive_gears`	int	Número de marchas
`drive_force`	double	Força do motor
`drive_max_flat_vel`	double	Velocidade de projeto em pista plana
`brake_force`, `handbrake_force`	double	Força de freio e de freio de mão
`steering_lock_deg`	double	Ângulo máximo de esterçamento (graus)
`traction_curve_max/min`	double	Aderência máxima/mínima dos pneus
`suspension_force`	double	Rigidez da suspensão
`*_damage_mult`	double	Multiplicadores de dano (colisão, deformação, motor)
`petrol_tank_volume`	double	Volume do tanque

6.5 Persistência (camada *processed*)

O conjunto é salvo em **Parquet** (preserva tipos e a coluna `flags` como lista) e em **CSV** (para inspeção rápida; `flags` vira texto separado por `|`).

```
saida_parquet = PROCESSED / "vehicles_clean.parquet"
saida_csv = PROCESSED / "vehicles_clean.csv"

df.coalesce(1).write.mode("overwrite").parquet(str(saida_parquet))
df.withColumn("flags", F.concat_ws("|", "flags")).toPandas().to_csv(saida_csv, index=False)

relido = spark.read.parquet(str(saida_parquet))
assert relido.count() == df.count() and relido.columns == df.columns
print(f"Parquet: {saida_parquet}  ({relido.count()} linhas x {len(relido.columns)} colunas)")
print(f"CSV    : {saida_csv}")
```

```
Parquet: ra1/data/processed/vehicles_clean.parquet  (790 linhas x 47 colunas)
CSV    : ra1/data/processed/vehicles_clean.csv
```

6.6 (Opcional) Copiar resultados para o Google Drive / baixar

O disco do Colab é temporário. Mude `COPIAR_PARA_DRIVE` para `True` para guardar os arquivos no seu Drive.

```
COPIAR_PARA_DRIVE = False

if COPIAR_PARA_DRIVE:
    import shutil
    from google.colab import drive
```

```
    drive.mount("/content/drive")
    destino_drive = Path("/content/drive/MyDrive/PjBL_RA1_GTAV")
    shutil.copytree(BASE / "data", destino_drive / "data", dirs_exist_ok=True)
    print("Copiado para", destino_drive)
```

RA1 / CONCLUSÃO

Conclusão do RA1

O conjunto `vehicles_clean` (790 veículos, 1 linha por modelo, chave `name`) integra os atributos de catálogo, desempenho e física (*handling*) e está pronto para as próximas etapas. Os principais problemas de qualidade (sentinelas, ausências disfarçadas, coluna *placeholder*, duplicatas de *handling*, inconsistência de caixa nas chaves e códigos de fabricante truncados) foram identificados, tratados e validados automaticamente.

Pontos de atenção:

- Comparar métricas **dentro de cada** `vehicle_type` (escalas muito diferentes entre carros e aeronaves).
- 5 veículos (`has_performance_stats = false`) devem ser excluídos de análises de desempenho, mas podem ser usados em contagens de catálogo.
- `dlc_name` é um código; para P5 será preciso associá-lo à data de lançamento de cada DLC.

O **RA2** (a partir da seção 7) usa o `vehicles_clean` como base para uma análise em *streaming*.

RA2 / 07

Visão geral e escolha do cenário

O projeto usa uma **base estática** (catálogo de veículos extraído do jogo), portanto seguimos o **Cenário A**: simulamos a chegada de novos registros, processamos com **Spark Structured Streaming** e agregamos por **janelas temporais**.

Item	Decisão
Cenário	**A** (dados estáticos, chegada simulada)
Fonte do stream	Arquivos JSON de **compras de veículos** que chegam em uma pasta, em lotes, em momentos diferentes
Tipo de janela	***Tumbling*** de **10 minutos** (sem deslocamento, janelas não sobrepostas)
Agregação	contagem de compras, valor total, valor médio e maior valor por janela

RA2 / 08

Objetivo da análise

O que queremos monitorar: a **demanda** e o **valor movimentado** em uma loja de veículos do jogo ao longo do tempo. A cada 10 minutos, quantas compras ocorreram e qual foi a soma do `monetary_value` dos veículos comprados?

Por que isso se liga ao RA1: a pergunta **P2** relaciona valor monetário e veículos. No RA2 trazemos o `monetary_value` do catálogo limpo para um cenário em tempo real: como o **valor movimentado** muda conforme novas compras chegam e conforme o **perfil dos veículos comprados** muda (por exemplo, uma procura por veículos de alto valor eleva o total da janela).

Pergunta do streaming: *em qual janela de tempo o valor movimentado foi maior, e como o resultado de cada janela muda à medida que novos registros chegam, inclusive registros atrasados?*

RA2 / 09

Fonte e processamento dos registros

Fonte (simulada). Cada registro é uma compra: `event_id`, `event_time` (momento da compra), `vehicle_name`, `vehicle_class`, `vehicle_type` e `valor`. Os veículos e valores **não são inventados**: são sorteados do `vehicles_clean` (`valor` = `monetary_value` do catálogo) (apenas `CAR` com `monetary_value` e estatísticas de desempenho), então os registros são coerentes com o projeto. Só os momentos e a escolha das compras são artificiais (semente fixa, para o resultado ser reproduzível).

Chegada em momentos diferentes. Os registros são divididos em **5 lotes**. Cada lote vira **um arquivo JSON** gravado em uma pasta enquanto a *query* de streaming está rodando. O Spark detecta o arquivo novo e processa só o que chegou.

> ***Nota sobre o valor:*** *no catálogo, `monetary_value` é um valor por faixas (por exemplo, 25 mil, 35 mil, 50 mil, 150 mil), e não um preço de loja detalhado. Por isso, nas tabelas usamos o termo* ***valor*** *(e não "preço" ou "gasto").*

Lote	Horário dos eventos	Observação
1	20:00 a 20:09	compras variadas
2	20:10 a 20:19	compras variadas
3	20:20 a 20:29	**pico de procura por veículos de alto valor** (`monetary_value` ≥ 100.000)
4	20:30 a 20:39 + 2 atrasados	2 eventos **atrasados** de 20:00 a 20:09 chegam junto
5	20:40 a 20:49	compras variadas

9.1 Geração dos lotes simulados

```
import json, random, shutil
from datetime import datetime, timedelta
from pathlib import Path

spark.conf.set("spark.sql.shuffle.partitions", "2")   # dados pequenos: evita 200 partições vazias

# Pool de veículos reais vindos do RA1 (carros com valor e desempenho conhecidos)
pool = [r.asDict() for r in (
    vehicles_clean
    .filter((F.col("vehicle_type") == "CAR") & F.col("monetary_value").isNotNull() &
F.col("has_performance_stats"))
    .select("name", "vehicle_class", "vehicle_type", "monetary_value")
    .orderBy("name")
    .collect()
)]
print("Veículos disponíveis para a simulação:", len(pool))

BASE_TEMPO = datetime(2026, 10, 20, 20, 0, 0)   # início da simulação (UTC)
rng = random.Random(42)                          # semente fixa: resultado reproduzível

# (n_eventos, minuto_inicial, minuto_final, valor_mínimo ou None)
PLANO_LOTES = {
    1: [(8, 0, 9, None)],
    2: [(6, 10, 19, None)],
    3: [(7, 20, 29, 100000)],                   # pico de procura por veículos de alto valor
    4: [(4, 30, 39, None), (2, 0, 9, None)],     # 2 registros atrasados (janela 20:00)
    5: [(5, 40, 49, None)],
}

def gerar_lote(num):
    eventos = []
    for n, ini, fim, valor_min in PLANO_LOTES[num]:
        candidatos = [v for v in pool if valor_min is None or v["monetary_value"] >= valor_min]
        for _ in range(n):
```

```
            v = rng.choice(candidatos)
            t = BASE_TEMPO + timedelta(minutes=rng.randint(ini, fim), seconds=rng.randint(0, 59))
            eventos.append({
                "event_id": f"L{num}-{len(eventos) + 1:02d}",
                "event_time": t.strftime("%Y-%m-%d %H:%M:%S"),
                "vehicle_name": v["name"],
                "vehicle_class": v["vehicle_class"],
                "vehicle_type": v["vehicle_type"],
                "valor": int(v["monetary_value"]),
            })
    return eventos

lotes = {i: gerar_lote(i) for i in PLANO_LOTES}
print("Lotes:", {i: len(e) for i, e in lotes.items()}, "| total =", sum(len(e) for e in
lotes.values()))
print("Exemplo de registro:", lotes[1][0])
```

```
Veículos disponíveis para a simulação: 597
Lotes: {1: 8, 2: 6, 3: 7, 4: 6, 5: 5} | total = 32
Exemplo de registro: {'event_id': 'L1-01', 'event_time': '2026-10-20 20:00:47', 'vehicle_name':
'comet7', 'vehicle_class': 'SPORT', 'vehicle_type': 'CAR', 'valor': 150000}
```

RA2 / 10

Janela temporal e agregação

- **Tipo:** *tumbling window* (janelas fixas, consecutivas e sem sobreposição).
- **Duração:** 10 minutos (`window("event_time", "10 minutes")`). Não há deslocamento, pois a janela *tumbling* não usa *slide*.
- **Tempo usado:** o **tempo do evento** (`event_time`, momento da compra), não o horário em que o Spark recebeu o arquivo. Por isso um registro atrasado cai na janela certa.
- **Agregações por janela:** `count` (nº de compras), `sum(valor)` (valor total), `avg(valor)` (valor médio) e `max(valor)` (maior valor).
- **Modo de saída:** `complete`: a cada lote, a tabela inteira de janelas é reescrita, e mostra como cada janela **evolui**.
- **Sem *watermark*:** de propósito, para que os registros atrasados atualizem a janela antiga. Em produção usaríamos *watermark* para limitar o estado guardado.
- **Gatilho:** a query é consumida com `processAllAvailable()` depois de cada lote, para a demonstração ser determinística.

10.1 Definição do stream e da query

```
from pyspark.sql.streaming import StreamingQuery

spark.conf.set("spark.sql.shuffle.partitions", "2")   # dados pequenos: evita 200 partições vazias

STREAM_DIR = BASE / "stream" / "entrada"          # pasta que o Spark monitora
CHECKPOINT = BASE / "stream" / "checkpoint"
TMP_DIR    = BASE / "stream" / "tmp"
for p in (STREAM_DIR, CHECKPOINT, TMP_DIR):
    shutil.rmtree(p, ignore_errors=True)
    p.mkdir(parents=True, exist_ok=True)

# Streaming exige schema explícito
schema_compras = T.StructType([
    T.StructField("event_id",      T.StringType()),
    T.StructField("event_time",    T.StringType()),
    T.StructField("vehicle_name",  T.StringType()),
    T.StructField("vehicle_class", T.StringType()),
    T.StructField("vehicle_type",  T.StringType()),
    T.StructField("valor",         T.LongType()),
])

# Fonte: lê cada arquivo novo da pasta, 1 arquivo (= 1 lote) por micro-batch
compras = (
    spark.readStream
    .schema(schema_compras)
    .option("maxFilesPerTrigger", 1)
    .json(str(STREAM_DIR))
    .withColumn("event_time", F.to_timestamp("event_time"))
)

# Janela tumbling de 10 minutos sobre o tempo do evento
janelas = (
    compras
    .groupBy(F.window("event_time", "10 minutes").alias("janela"))
    .agg(
        F.count("*").alias("compras"),
        F.sum("valor").alias("valor_total"),
        F.round(F.avg("valor")).cast("long").alias("valor_medio"),
        F.max("valor").alias("maior_valor"),
    )
)

query = (
    janelas.writeStream
```

```
    .format("memory")                # resultado consultável como tabela "janelas_10min"
    .queryName("janelas_10min")
    .outputMode("complete")
    .option("checkpointLocation", str(CHECKPOINT))
    .start()
)
print("Query ativa:", query.isActive, "| é streaming:", compras.isStreaming)
```

```
26/10/21 01:03:41 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming
DataFrames/Datasets and will be disabled.
Query ativa: True | é streaming: True
```

RA2 / 11

Execução: chegada dos lotes e resultados

Para cada lote: (1) o arquivo é gravado na pasta monitorada **com a query já rodando**; (2) o Spark processa o que chegou; (3) mostramos o **nº de registros lidos naquele micro-batch** (evidência de que só o novo foi processado) e a **tabela de janelas atualizada**.

```
import time

def formatar(df):
    # Mostra a janela como intervalo legível
    return (df.select(
                F.date_format(F.col("janela.start"), "HH:mm").alias("inicio"),
                F.date_format(F.col("janela.end"), "HH:mm").alias("fim"),
                "compras", "valor_total", "valor_medio", "maior_valor")
              .orderBy("inicio"))

def enviar_lote(num):
    # Grava em pasta temporária e move: o Spark nunca vê arquivo pela metade
    tmp = TMP_DIR / f"lote_{num}.json"
    tmp.write_text("\n".join(json.dumps(e) for e in lotes[num]), encoding="utf-8")
    shutil.move(str(tmp), STREAM_DIR / f"lote_{num}.json")

historico = {}   # guarda o resultado após cada lote, para a análise final

for num in PLANO_LOTES:
    print(f"\n{'=' * 70}\nLOTE {num}: chegaram {len(lotes[num])} novos registros")
    enviar_lote(num)
    query.processAllAvailable()          # espera o Spark processar o que chegou

    progresso = [p for p in query.recentProgress if p["numInputRows"] > 0][-1]
    print(f"micro-batch #{progresso['batchId']} -> registros lidos neste batch:
{progresso['numInputRows']}")

    resultado = formatar(spark.table("janelas_10min"))
    historico[num] = [r.asDict() for r in resultado.collect()]
    resultado.show(truncate=False)
    time.sleep(1)
```

```
======================================================================
LOTE 1: chegaram 8 novos registros
micro-batch #0 -> registros lidos neste batch: 8
+------+-----+-------+-----------+-----------+-----------+
|inicio|fim  |compras|valor_total|valor_medio|maior_valor|
+------+-----+-------+-----------+-----------+-----------+
|20:00 |20:10|8      |650000     |81250      |150000     |
+------+-----+-------+-----------+-----------+-----------+

======================================================================
LOTE 2: chegaram 6 novos registros
micro-batch #1 -> registros lidos neste batch: 6
+------+-----+-------+-----------+-----------+-----------+
|inicio|fim  |compras|valor_total|valor_medio|maior_valor|
+------+-----+-------+-----------+-----------+-----------+
|20:00 |20:10|8      |650000     |81250      |150000     |
|20:10 |20:20|6      |405000     |67500      |150000     |
+------+-----+-------+-----------+-----------+-----------+

======================================================================
LOTE 3: chegaram 7 novos registros
micro-batch #2 -> registros lidos neste batch: 7
+------+-----+-------+-----------+-----------+-----------+
|inicio|fim  |compras|valor_total|valor_medio|maior_valor|
+------+-----+-------+-----------+-----------+-----------+
|20:00 |20:10|8      |650000     |81250      |150000     |
|20:10 |20:20|6      |405000     |67500      |150000     |
|20:20 |20:30|7      |1050000    |150000     |150000     |
+------+-----+-------+-----------+-----------+-----------+
```

```
======================================================================
LOTE 4: chegaram 6 novos registros
micro-batch #3 -> registros lidos neste batch: 6
+------+-----+-------+-----------+-----------+-----------+
|inicio|fim  |compras|valor_total|valor_medio|maior_valor|
+------+-----+-------+-----------+-----------+-----------+
|20:00 |20:10|10     |835000     |83500      |150000     |
|20:10 |20:20|6      |405000     |67500      |150000     |
|20:20 |20:30|7      |1050000    |150000     |150000     |
|20:30 |20:40|4      |215000     |53750      |100000     |
+------+-----+-------+-----------+-----------+-----------+

======================================================================
LOTE 5: chegaram 5 novos registros
micro-batch #4 -> registros lidos neste batch: 5
+------+-----+-------+-----------+-----------+-----------+
|inicio|fim  |compras|valor_total|valor_medio|maior_valor|
+------+-----+-------+-----------+-----------+-----------+
|20:00 |20:10|10     |835000     |83500      |150000     |
|20:10 |20:20|6      |405000     |67500      |150000     |
|20:20 |20:30|7      |1050000    |150000     |150000     |
|20:30 |20:40|4      |215000     |53750      |100000     |
|20:40 |20:50|5      |267000     |53400      |150000     |
+------+-----+-------+-----------+-----------+-----------+
```

11.1 Resultado final e verificação

```python
print("Resultado final por janela de 10 minutos:")
formatar(spark.table("janelas_10min")).show(truncate=False)

total_stream = spark.table("janelas_10min").agg(F.sum("compras")).first()[0]
total_valor  = spark.table("janelas_10min").agg(F.sum("valor_total")).first()[0]
total_gerado = sum(len(e) for e in lotes.values())
valor_gerado = sum(e_["valor"] for e in lotes.values() for e_ in e)

# O streaming não perdeu nem duplicou nenhum registro
assert total_stream == total_gerado, (total_stream, total_gerado)
assert total_valor == valor_gerado
print(f"OK: {total_stream} registros processados (= {total_gerado} gerados) | valor total
{total_valor:,}")

query.stop()
print("Query encerrada:", not query.isActive)
```

```
Resultado final por janela de 10 minutos:
+------+-----+-------+-----------+-----------+-----------+
|inicio|fim  |compras|valor_total|valor_medio|maior_valor|
+------+-----+-------+-----------+-----------+-----------+
|20:00 |20:10|10     |835000     |83500      |150000     |
|20:10 |20:20|6      |405000     |67500      |150000     |
|20:20 |20:30|7      |1050000    |150000     |150000     |
|20:30 |20:40|4      |215000     |53750      |100000     |
|20:40 |20:50|5      |267000     |53400      |150000     |
+------+-----+-------+-----------+-----------+-----------+

OK: 32 registros processados (= 32 gerados) | valor total 2,772,000
Query encerrada: True
```

11.2 Evolução de cada janela entre os lotes

```python
# Quantas compras cada janela tinha após cada lote (mostra o efeito dos dados novos e dos atrasados)
inicios = sorted({r["inicio"] for linhas in historico.values() for r in linhas})
evolucao = []
for ini in inicios:
    linha = [f"{ini}"]
    for num in PLANO_LOTES:
        r = next((x for x in historico[num] if x["inicio"] == ini), None)
        linha.append(f"{r['compras']}" if r else "-")
```

```
    evolucao.append(tuple(linha))
tabela(evolucao, ["janela_inicio"] + [f"apos_lote_{n}" for n in PLANO_LOTES])
```

```
+-------------+-----------+-----------+-----------+-----------+-----------+
|janela_inicio|apos_lote_1|apos_lote_2|apos_lote_3|apos_lote_4|apos_lote_5|
+-------------+-----------+-----------+-----------+-----------+-----------+
|20:00        |8          |8          |8          |10         |10         |
|20:10        |-          |6          |6          |6          |6          |
|20:20        |-          |-          |7          |7          |7          |
|20:30        |-          |-          |-          |4          |4          |
|20:40        |-          |-          |-          |-          |5          |
+-------------+-----------+-----------+-----------+-----------+-----------+
```

RA2 / 12

Resultados e interpretação

O que foi observado (valores da execução com semente 42; ao mudar a semente, os números mudam, mas o comportamento é o mesmo):

1. **O Spark processou só o que chegou.** A cada lote, o `numInputRows` do micro-batch foi igual ao tamanho do lote (8, 6, 7, 6 e 5 registros). Nenhum arquivo foi relido, e o total final (32 registros, valor total 2.772.000) bate com o que foi gerado, sem perda nem duplicação.
2. **Novas janelas aparecem conforme o tempo avança.** Depois do lote 1 existia só a janela 20:00; cada lote seguinte abriu uma janela nova (20:10, 20:20, 20:30 e 20:40). As janelas já fechadas não mudaram quando chegaram dados de horários posteriores.
3. **O pico de alto valor é visível na janela 20:20.** Foram 7 compras, e todas de veículos de alto valor: o valor total foi de **1.050.000**, mais que o dobro da janela 20:10 (405.000) com apenas uma compra a mais (6 contra 7). O valor médio subiu para 150.000, contra 67.500 na janela anterior. A **contagem sozinha não mostraria esse pico**: o que o revela é a soma do `monetary_value`, o que justifica agregar mais de uma medida.
4. **Registros atrasados corrigem a janela antiga.** No lote 4 chegaram 2 eventos de 20:00 a 20:09. Como a janela usa o **tempo do evento** e a saída é `complete`, a janela 20:00 foi **reescrita**: de 8 para 10 compras e de 650.000 para 835.000 de valor total. As janelas 20:10 e 20:20 não mudaram. A tabela de evolução (seção 11.2) mostra isso: a contagem da janela `20:00` passa de 8 para 10 no lote 4.
5. **Após o pico, o movimento volta ao normal.** As janelas 20:30 e 20:40 têm valor médio de cerca de 53 mil, bem abaixo das janelas 20:00 (83.500) e 20:20 (150.000).

Interpretação: em um monitoramento em tempo real, o resultado de uma janela **não é definitivo**: ele pode mudar com dados atrasados. Isso é esperado em *streaming* e é o motivo de existirem *watermarks* em sistemas reais, que definem até quando se espera por dados atrasados. Aqui o foco foi mostrar o comportamento básico, então a janela fica aberta para sempre. As conclusões valem para a **simulação**: não dizem nada sobre compras reais do jogo, e `monetary_value` é um valor por faixas do catálogo.

RA2 / CONCLUSÃO

Conclusão do RA2

Foi implementada uma análise em *streaming* com **Spark Structured Streaming**: registros de compra simulados, mas construídos a partir do catálogo real do RA1, chegam em arquivos separados e são processados incrementalmente. Os resultados são agregados em **janelas *tumbling* de 10 minutos** (contagem, valor total, valor médio e maior valor). A validação final confirma que **nenhum registro foi perdido ou duplicado**. Os resultados mostram que o resultado de uma janela **muda quando chegam dados novos** e que registros **atrasados** corrigem janelas antigas, porque a agregação usa o tempo do evento.

Limitações: os eventos são simulados, então os valores de demanda não representam compras reais do jogo; foi usada uma única janela e nenhum *watermark*, o que seria necessário em produção para limitar o estado guardado.